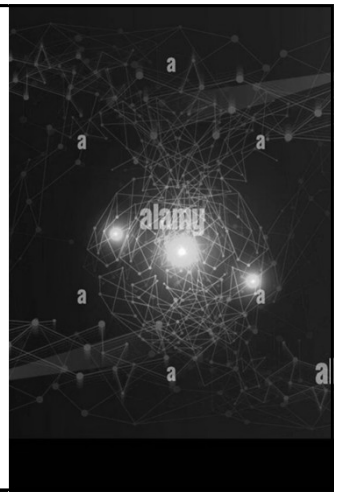
Understanding Stacks and Queues in Algorithms



1

Chapter 1

What is a Stack? The LIFO Principle

- ***LIFO (Last In, First Out):*** *The element most recently added is the first one to be removed. This principle dictates the behavior of the data structure.*
- ***A Stack of Plates Analogy:*** *Visualize a physical stack of plates—you can only place a new plate on top, and you can only take a plate from the top.*
- ***Core Operations Defined:***
  - *Push: Adds a new item to the top of the stack.*
  - *Pop: Removes the item from the top of the stack and returns it.*
  - *Peek/Top: Allows viewing the top element without removing it.*

Stacks are crucial for managing temporary data or sequences where reversal is needed.

2

Chapter 2

What is a Queue? The FIFO Principle

FIFO Order

First In, First Out (FIFO): *The element that was added first is always the first one to be processed and removed.*

The Ticket Line Analogy

Consider people waiting in a line for a ticket counter. The person who joined the line first is the person served first.

Core Operations Defined

Enqueue

Adding a new element to the ***rear*** *(end) of the queue.*

Dequeue

Removing the oldest element from the ***front*** *(head) of the queue.*

Front/Peek

Viewing the element at the front without removing it.

3

Key Differences

While both are linear data structures, their access patterns define their use cases and algorithmic behaviors.

Stack (LIFO)	Queue (FIFO)
Insertion and removal occur at a single end, typically called the ***Top****.*	*Insertion occurs at the* ***Rear****, and removal occurs at the* ***Front****.*
Order of processing is Last In, First Out.	*Order of processing is First In, First Out.*
Primary use in managing nested operations (e.g., function calls, undo history).	*Primary use in handling resources or tasks in order (e.g., scheduling, task pools).*

4

Real-World Applications of Stacks and Queues

Stacks: Program Execution

Manages function calls and local variables in programming—when a function is called, it's pushed onto the call stack; when it returns, it's popped.

Stacks: Undo/Redo Feature

Every action is stored in a stack; "Undo" pops the last action. This is common in text editors and graphic design software.

Queues: Task Scheduling

Operating systems use queues to manage processes waiting for CPU time or resources, ensuring fair execution based on arrival time.

Queues: BFS in Graphs

Breadth-First Search uses a queue to explore nodes level by level, finding the shortest path in unweighted graphs.

5

Implementation Overview: Achieving Efficiency

Both Stacks and Queues can be efficiently implemented using either arrays or linked lists, aiming for constant time complexity for core operations.

Linked List Implementation

- *Dynamically allocates memory, avoiding overflow.*
- *Stack: Pointers maintain the head (Top) for O(1) operations.*
- *Queue: Pointers track both the Head (Front) and Tail (Rear) for O(1) Enqueue/Dequeue.*

Array Implementation

- *Fixed size, potentially leading to overflow/underflow.*
- *Stack: Uses a simple array and an index (Top) for O(1) Push/Pop.*
- *Queue: Often uses a **Circular Buffer** to efficiently manage front and rear pointers within the fixed array space.*

Both structures support O(1) time complexity for their main operations (Push/Pop, Enqueue/Dequeue), ensuring high performance regardless of the data size.

BASIS FOR COMPARISON	ARRAY	LINKED LIST
asic	It is a consistent set of a fixed number of data items.	It is an ordered set comprising a variable number of data items.
ize	Specified during declaration.	No need to specify; grow and shrink during executio
torage Allocation	Element location is allocated during compile time.	Element position is assigne during run time.
rder of the ements	Stored consecutively	Stored randomly
ccessing the ement	Direct or randomly accessed, i.e., Specify the array index or subscript.	Sequentially accessed, i.e. Traverse starting from the first node in the list by the pointer.
nsertion and eletion of ement	Slow relatively as shifting is required.	Easier, fast and efficient.
earching	Binary search and linear search	linear search
emory required	less	More
emory Utilization	Ineffective	Efficient

6

Simple Stack Example in Code (Conceptual)

This conceptual pseudo-code demonstrates the LIFO behavior using the standard Push and Pop methods.

```
class Stack:    def __init__(self):        self.items = []    def push(self, item):       #
LIFO: Adding element to the end (top)        self.items.append(item)        print(f"Pushed:
{item}")    def pop(self):        # LIFO: Removing element from the end (top)        if not
self.is_empty():            return self.items.pop()        return "Stack is empty"    def
is_empty(self):        return len(self.items) == 0# Demonstrationmy_stack =
Stack()my_stack.push('Task 1')  # Last Inmy_stack.push('Task 2')print(f"Popped:
{my_stack.pop()}")  # First Out: Task 2print(f"Popped: {my_stack.pop()}")
```

Initial Stack
Empty: []

Push 'Task 1'
Stack: ['Task 1']

Push 'Task 2'
Stack: ['Task 1', 'Task 2']

Pop Operation
Returns 'Task 2' (LIFO)

7